

SONICLINK

Acoustic FSK Communication System

PROJECT REPORT

Java • BFSK • PCM • CRC • AcousticCommunication

Prepared by Gunjan
2026

A working prototype that turns digital data into sound, sends it acoustically, and reconstructs it at the receiver.

Table of Contents

1. Introduction		3
2. Problem Statement	.	4
3. Functional		5
Requirements	.	6
		7
4. Non-Functional	.	9
Requirements 5.		1
System Architecture	.	3
		1
6. Design Diagrams	.	5
		
7. Design Decisions &		1
Rationale 8.	.	8
		
Implementation Details	.	2
		0
9. Screenshots /		2
Results	.	1
		
10. Testing Approach	.	2
		
11. Challenges Faced		2
	.	2
12. Learnings		3
	.	2
13. Future		2
Enhancements	.	4
		
14. Conclusion	.	2
		
15. References	.	5
		
	.	
		
	.	

1. Introduction

SonicLink is a Java-based acoustic communication prototype. It explores whether ordinary audio hardware can carry structured digital information by using Binary Frequency Shift Keying (BFSK), packet framing, audio I/O, demodulation, and CRC validation.

The system accepts a message or supported file input, converts the data into a packet, maps binary values to audio frequencies, generates PCM samples, and plays the waveform. A receiving path records microphone samples, detects the preamble, identifies the transmitted frequencies, reconstructs the packet, and checks its integrity.

1.1 Project Snapshot

Item	Value
Language	Java
Modulation	BFSK
Bit 0	1200 Hz
Bit 1	2200 Hz
Medium	Acoustic audio
Integrity	CRC
Interface	CLI

2. Problem Statement

Modern communication normally depends on Wi-Fi, Bluetooth, mobile networks, or physical cabling. SonicLink investigates an intentionally simple alternative: transmitting information through audible or near-audible tones. The engineering problem is not only generating a tone, but synchronizing the receiver, preserving packet boundaries, recovering bits from audio, and checking whether the reconstructed data is valid.

The project therefore treats sound as a communication channel and implements the full path from application data to physical audio and back to application data.

2.1 Scope

The scope includes text transmission, packet framing, FSK modulation/demodulation, audio capture/playback, preamble detection, CRC validation, a CLI, and file-input processing for supported formats.

3. Functional Requirements

3.1 Message Transmission

Accept a command-line message, encode it as UTF-8 payload data, create a packet, modulate the packet, and transmit the resulting PCM audio.

3.2 File Input

Accept a supported file path and process its text content for transmission. PDF/image input may use local extraction/OCR tools.

3.3 Audio Reception

Capture microphone audio for a configured duration and search for a SonicLink preamble.

3.4 Demodulation

Identify the configured FSK frequencies and reconstruct the transmitted bit stream.

3.5 Packet Decoding

Decode packet metadata and payload according to the SonicLink protocol.

3.6 Integrity Verification

Validate CRC information and report whether the received payload passes the integrity check.

3.7 CLI

Expose send, send-file, listen, self-test, and help operations.

4. Non-Functional Requirements

4.1 Usability

Commandss should be simple enough to demonstrate from a terminal without requiring a GUI.

4.2 Maintainability

Audio,codec,packet,utility, and CLI responsibilities should remain separated into dedicated classes.

4.3 Portability

Corecommunicationfunctionality should rely primarily on Java and the Java Sound API.

4.4 Reliability

Synchronizationand CRC checking should reduce errors caused by misalignment or corrupted data.

4.5 Observability

CLloutputshouldexpose useful information such as mode, payload size, protocol version, and status.

5. System Architecture

The architecture separates application-level data handling from signal-level communication, allowing packet and codec logic to be tested without physical audio hardware.

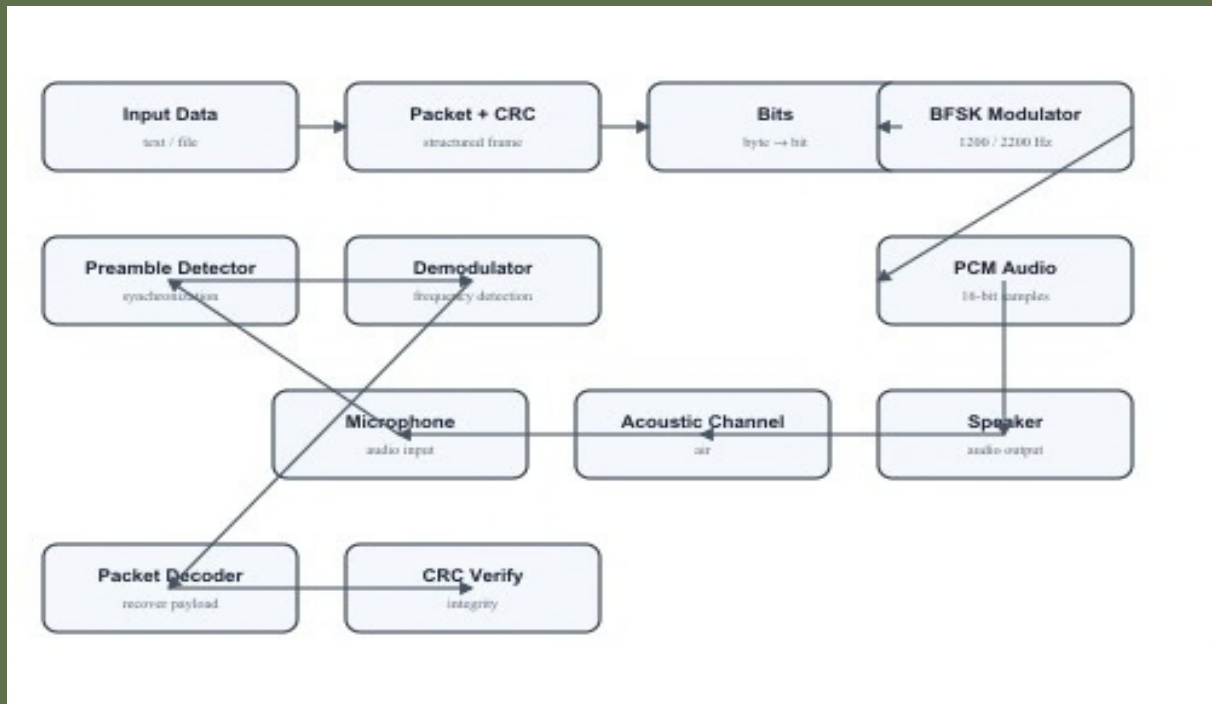


Figure 5.1. SanctiLink end-to-end architecture.

5.1 Communication Pipeline

Input Data → Packet + CRC → Bits → BFSK → PCM → Speaker
 Speaker → Acoustic Channel → Microphone → Preamble → Demodulator
 → Packet Decoder → CRC Verify Output

6. Design Diagrams

6.1 Workflow Diagram

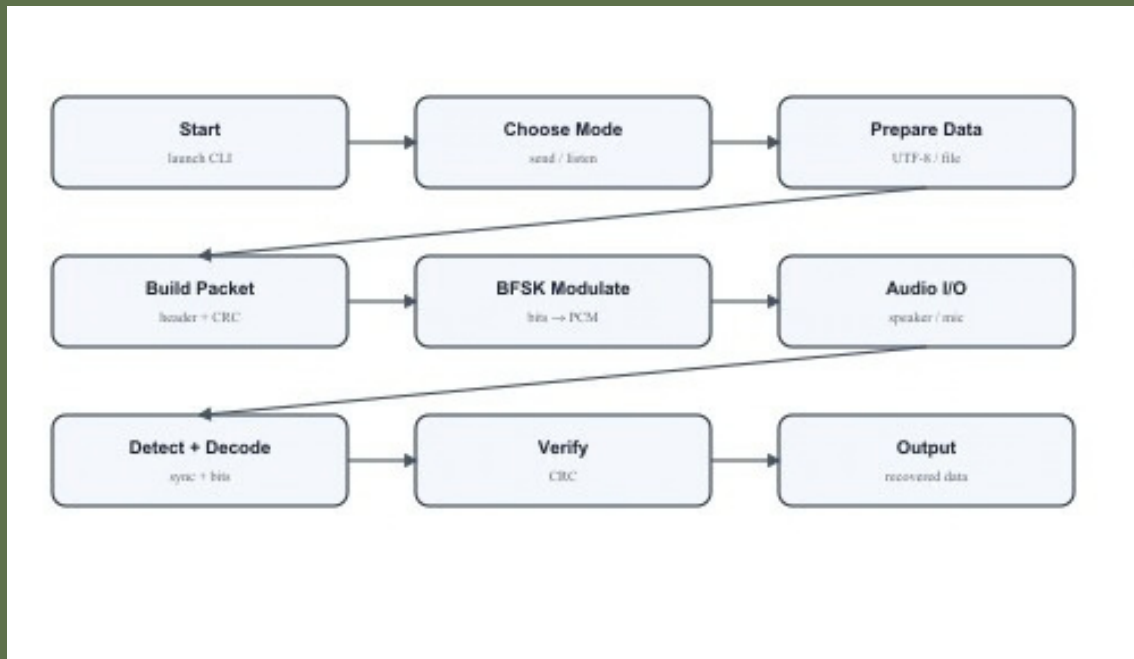


Figure 6.1: User-to-output workflow

6.2 Use Case Diagram

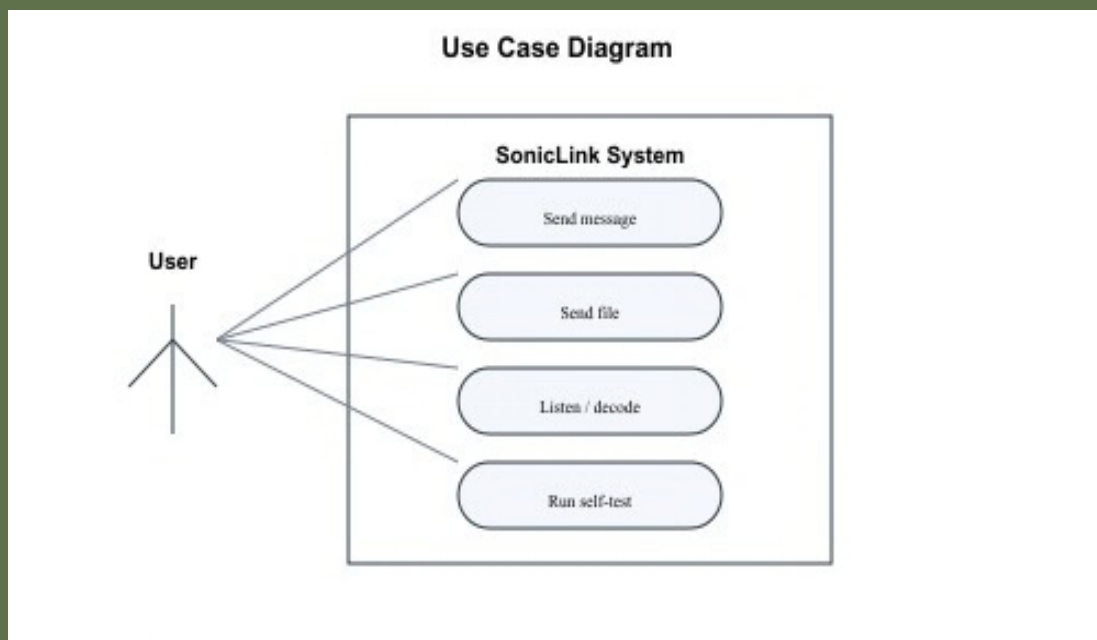


Figure 6.2: Primary CLI use cases

6.3 Class / Component Diagram

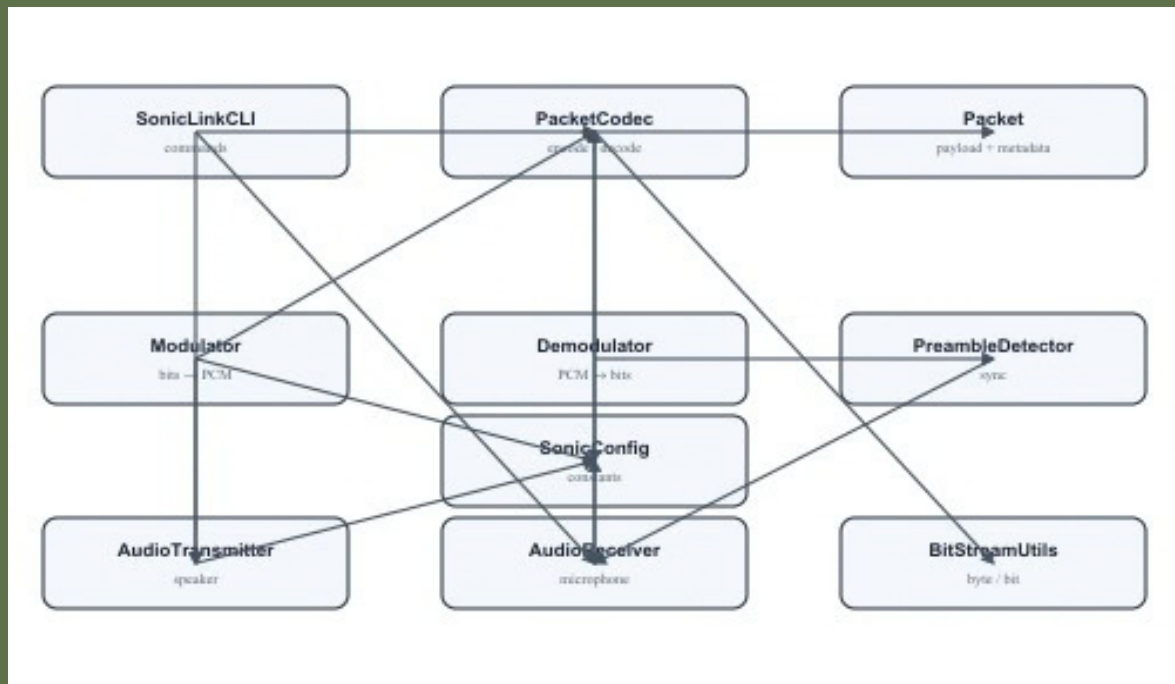


Figure 6.3. Main software components and relationships.

6.4 Sequence Diagram

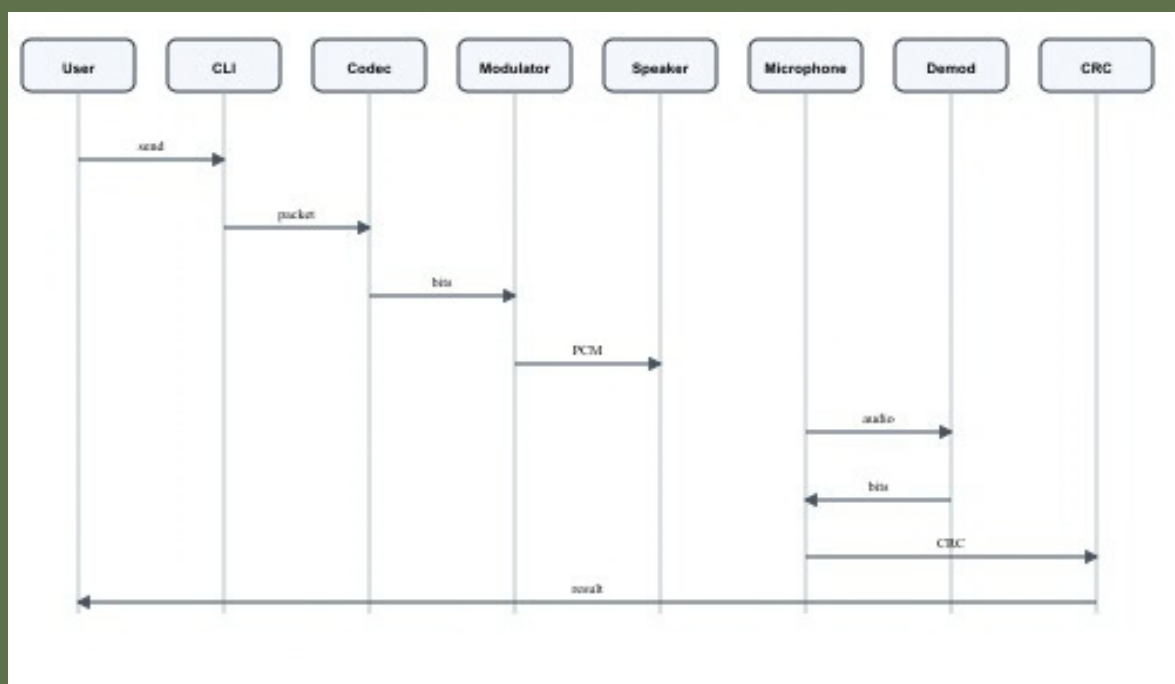


Figure 6.4. Representative send-to-recv sequence.

7. Design Decisions & Rationale

7.1 Why BFSK

BFSK provides a straightforward mapping between binary data and two frequencies. It is easy to generate with sampled audio and clear to demonstrate.

7.2 Why a Preamble

A receiver needs a synchronization point before interpreting symbols. A dedicated preamble provides a packet boundary.

7.3 Why CRC

Audio transmission can introduce errors. CRC gives the receiver a compact mechanism for detecting corruption.

7.4 Why Packetization

Structured packets provide protocol information, payload length, payload data, and integrity information.

7.5 Why Java

Java provides object-oriented organization, Java Sound API access, file handling, and convenient command-line execution.

7.6 Frequency Detection

Known target frequencies allow focused energy detection, making the receiver practical for a prototype.

8. Implementation Details

8.1 Data Representation

Input text is represented as UTF-8 bytes. Packet bytes are converted into a boolean bit stream before modulation and reconstructed after demodulation.

8.2 Modulation

The modulator generates signed 16-bit PCM samples. Each binary symbol is represented by its configured frequency for the configured symbol duration.

8.3 Reception

AudioReceiver uses the Java Sound API to capture mono PCM microphone data. The CLI controls the recording duration.

8.4 Preamble Detection

PreambleDetector searches recorded samples for energy concentrated at the configured preamble frequency and uses a confidence threshold to reject weak matches.

8.5 Demodulation

The demodulator analyzes symbol windows and determines whether the 1200 Hz or 2200 Hz component dominates.

8.6 File Processing

TXT input can be read directly as UTF-8. PDF and image input can be routed through local extraction/OCR tools when installed.

8.7 Build Commands

```
bash build.sh
bash build.sh run send "Hello SonicLink"
bash build.sh run send-file ./test.txt
bash build.sh run listen
bash build.sh run self-test
```

9. Screenshots / Results

The panels below document the recorded command-line results available from the project workflow. They are intentionally presented as report-ready evidence panels. If the submission rubric requires literal original screenshots, replace the corresponding panels with the original terminal image captures.

```
gunjankumariyadav@Gunjans-MacBook-Air SonicLink % bash build.sh run send-  
file ./test.txt  
=====
```

Building SonicLink

```
=====
```

Compiling 27 Java source files...

```
✓ Build complete.  
✓ Class files located in out/  
=====
```

Sending TEXT file: 60 bytes in 1 chunk(s) (fileId=42599)
chunk 1/1 (67 bytes payload)

```
✓ File transmission complete.
```

```
✓ Build complete.  
✓ Class files located in out/  
=====
```

📡 Preparing SonicLink transmission...

Message: Hello SonicLink
Payload: 15 bytes
Packet: 21 bytes
Bits: 168
FSK: 1200.0 Hz = 0, 2200.0 Hz = 1
Symbol duration: 50 ms
Transmission duration: 8.40 seconds

📡 Transmitting...

```
✓ Transmission complete.
```

```
gunjankumariyadav@Gunjans-MacBook-Air SonicLink % bash build.sh run list  
n  
=====
```

Building SonicLink

```
=====
```

Compiling 27 Java source files...

```
✓ Build complete.  
✓ Class files located in out/  
=====
```

📡 SonicLink Receiver

```
=====
```

Listening for acoustic transmission...

```
🔊 Recording...  
✓ Recording complete.  
✓ Recording complete.  
PCM bytes: 1764000  
✓ Preamble detected.  
Data starts at byte: 478162  
🔊 Decoding packet header...  
✓ Header received.  
Payload length: 15 bytes  
🔊 Demodulating packet...  
✓ Demodulation complete.  
Bits received: 168  
=====
```

✓ PACKET RECEIVED SUCCESSFULLY

```
=====
```

Message: Hello SonicLink
Payload: 15 bytes
Protocol version: 1
✓ CRC verified
=====

SonicLink • Project Report

Page 12

9. Screenshots / Results

The panels below document the recorded command-line results available from the project workflow. They are intentionally presented as report-ready evidence panels. If the submission rubric requires literal original screenshots, replace the corresponding panels with the original terminal image captures.

```
bash build.sh run listen

● gunjankumariyadav@Gunjans-MacBook-Air SonicLink % bash build.sh run send
"Hello SonicLink"
=====
                        Building SonicLink
=====
Compiling 27 Java source files...

✓ Build complete.
✓ Class files located in out/
=====

📡 Preparing SonicLink transmission...
  Message: Hello SonicLink
  Payload: 15 bytes
  Packet: 21 bytes
  Bits: 168
  FSK: 1200.0 Hz = 0, 2200.0 Hz = 1
  Symbol duration: 50 ms
  Transmission duration: 8.40 seconds

📡 Transmitting...
✓ Transmission complete.
```

```
Transmission complete.

● gunjankumariyadav@Gunjans-MacBook-Air SonicLink % echo "Hello from SonicL
ink file transfer!" > test.txt
● gunjankumariyadav@Gunjans-MacBook-Air SonicLink %
```

```
✓ Build complete.
✓ Class files located in out/
=====

📡 Preparing SonicLink transmission...
  Message: Hello SonicLink
  Payload: 15 bytes
  Packet: 21 bytes
  Bits: 168
  FSK: 1200.0 Hz = 0, 2200.0 Hz = 1
  Symbol duration: 50 ms
  Transmission duration: 8.40 seconds

📡 Transmitting...
✓ Transmission complete.
```

10. Testing Approach

10.1 Unit-Level Testing

Focused tests cover bit conversion, frequency detection, checksum validation, and packet encoding/decoding.

10.2 Software Loopback

A software loopback passes generated packet bytes through modulation and demodulation without physical audio hardware, validating the signal-processing pipeline.

10.3 Acoustic Test

The transmitter was exercised through actual audio output, and the generated FSK signal was audibly present at the speaker.

10.4 File Test

A TXT file was created and passed to send-file. The observed run reached transmission completion, confirming the file-input path reached the working transmitter.

11. Challenges Faced

11.1 Synchronization

The receiver needs a known starting point, so preamble detection was required.

11.2 Signal Confidence

Frequency scoring can behave poorly on silence or weak signals, so low-energy handling and confidence thresholds are used.

11.3 Audio Timing

Microphone and speaker operations are real-time operations, making recording duration and symbol timing important.

11.4 File Integration

The file-processing layer had to be connected to the existing packet and modulation pipeline without disturbing the working codec.

11.5 Build Integration

New Java classes must be included in the build source tree, making the final compilation a necessary integration check.

12. Learnings

The project demonstrated the difference between representing information digitally and transporting it through a physical signal. Reliable transmission requires packet boundaries, timing, synchronization, signal detection, and integrity checks.

The implementation also reinforced Java modularity. Separating `Packet`, `PacketCodec`, `Modulator`, `Demodulator`, `PreambleDetector`, `AudioReceiver`, `AudioTransmitter`, and `CLI` responsibilities makes debugging more manageable.

A key practical lesson was that successful compilation does not guarantee successful hardware communication. Acoustic testing therefore complements software testing.

13. Future Enhancements

- Adaptive frequency selection based on measured noise.
- Forward error correction in addition to CRC.
- Multi-packet file transfer with sequence numbers and reassembly.
- Direct binary file reconstruction for arbitrary files.
- Automatic gain/noise calibration.
- Continuous listening mode with packet-triggered capture.
- Real-time waveform and spectrum visualization.
- GUI-based sender and receiver.
- Improved synchronization and collision handling.
- Performance measurements for range, bit rate, and packet error rate.

14. Conclusion

SonicLink demonstrates a complete acoustic digital communication prototype using Java. The system transforms application data into binary information, represents those bits with two audio frequencies, transmits the resulting waveform, and provides a receiving path that can synchronize, demodulate, decode, and verify the packet.

The project exposes several engineering layers that modern communication systems normally hide: framing, timing, signal generation, detection, error checking, and hardware I/O.

The working transmitter workflow provide a foundation for extending SonicLink toward stronger error correction, higher data rates, and true multi-packet binary file transfer.

15. References

- [JavaPlatformdocumentation](#) and [Java Sound API](#) concepts used for audio capture and playback.
- [SonicLink](#) project source code and build scripts.
- Project tests for packet encoding/decoding, CRC validation, bit conversion, and frequency detection.
- Goertzel algorithm concept for detecting energy at known target frequencies.
- Digital communication fundamentals covering Binary Frequency Shift Keying (BFSK).

Repository

GitHub repository: [kongunjan/SonicLink](#)

Submission Note

Verify that the final repository contains README.md and STATEMENT.md and that the submitted screenshots correspond to the final run on the submission machine.